

CS223 Computer Architecture & Organization

Dynamic Scheduling with Speculation



John Jose

Associate Professor

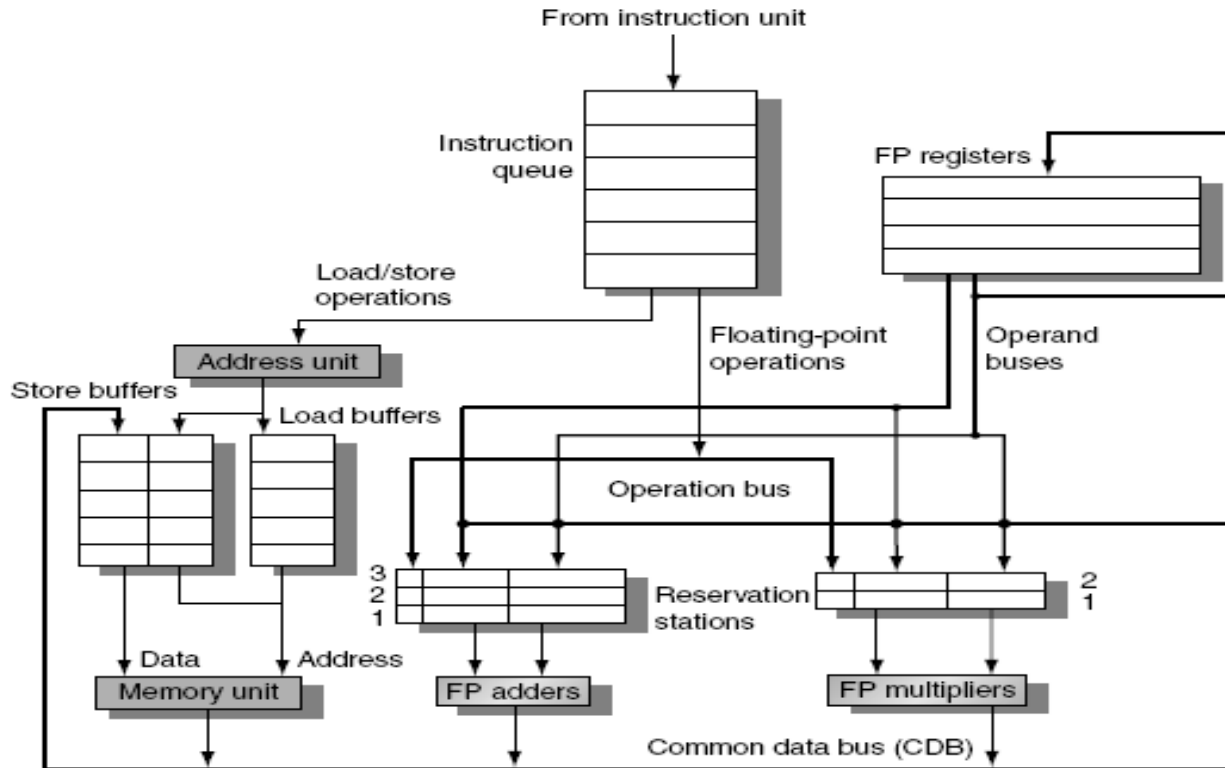
Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

How dynamic scheduling works ?

- ❖ To allow out-of-order execution, **split the ID stage into two**
 - ❖ **Issue**—Decode instructions, check for structural hazards.
 - ❖ **Read operands**—Wait until no data hazards, then read operands.
- ❖ In a dynamically scheduled pipeline, all instructions pass through the **issue stage in order** (in-order issue); however, they can be stalled or bypass each other in the second stage (read operands) and thus enter **execution out of order**.
- ❖ Done by - **Tomasulo's algorithm**

Tomasulo's Algorithm



Reservation Stations

❖ Each reservation station has seven fields.

❖ {Op, Qj, Qk, Vj, Vk, A, Busy}

1. **Op** —The operation to perform on source operands S1 and S2.
- 2,3. **Qj, Qk** —The reservation stations that will produce the corresponding source operand; a value of zero indicates that the source operand is already available in Vj or Vk.
- 4,5. **Vj, Vk** —The value of the source operands.

Only one of the **V field** or the **Q field** is valid for each operand.
For loads, the Vk field is used to hold the offset field.

Reservation Stations

- ❖ Each reservation station has seven fields.

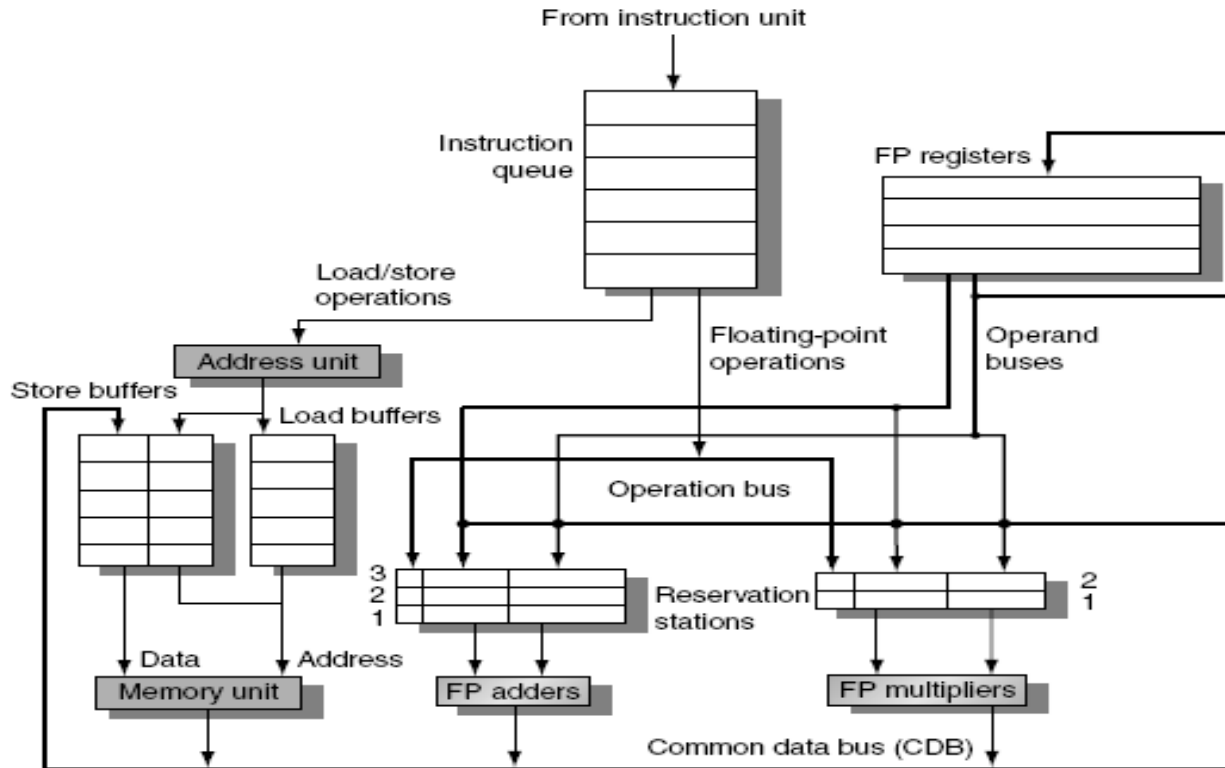
- ❖ {Op, Qj, Qk, Vj, Vk, A, Busy}

6. **A** —Used to hold information for the memory address calculation for a load or store. Initially, the immediate field of the instruction is stored here; after the address calculation, the effective address is stored here.
7. **Busy** —Indicates that this reservation station and its accompanying functional unit are occupied.

Register Status Indicator

- ❖ **The register file has a field, Q_i : (RSI)**
- ❖ Q_i —The number of the reservation station that contains the operation whose result should be stored into this register.
- ❖ If the value of $Q_i = 0$ no currently active instruction is computing a result destined for this register, meaning that the value is simply the register contents.

Tomasulo's Algorithm



Dynamic Scheduling - Tomasulo

❖ Issue

- ❖ Get next instruction from FIFO queue
- ❖ If RS available, issue the instruction to the RS with operand values if available
- ❖ If operand values not available, stall the instruction

Dynamic Scheduling - Tomasulo

❖ Execute

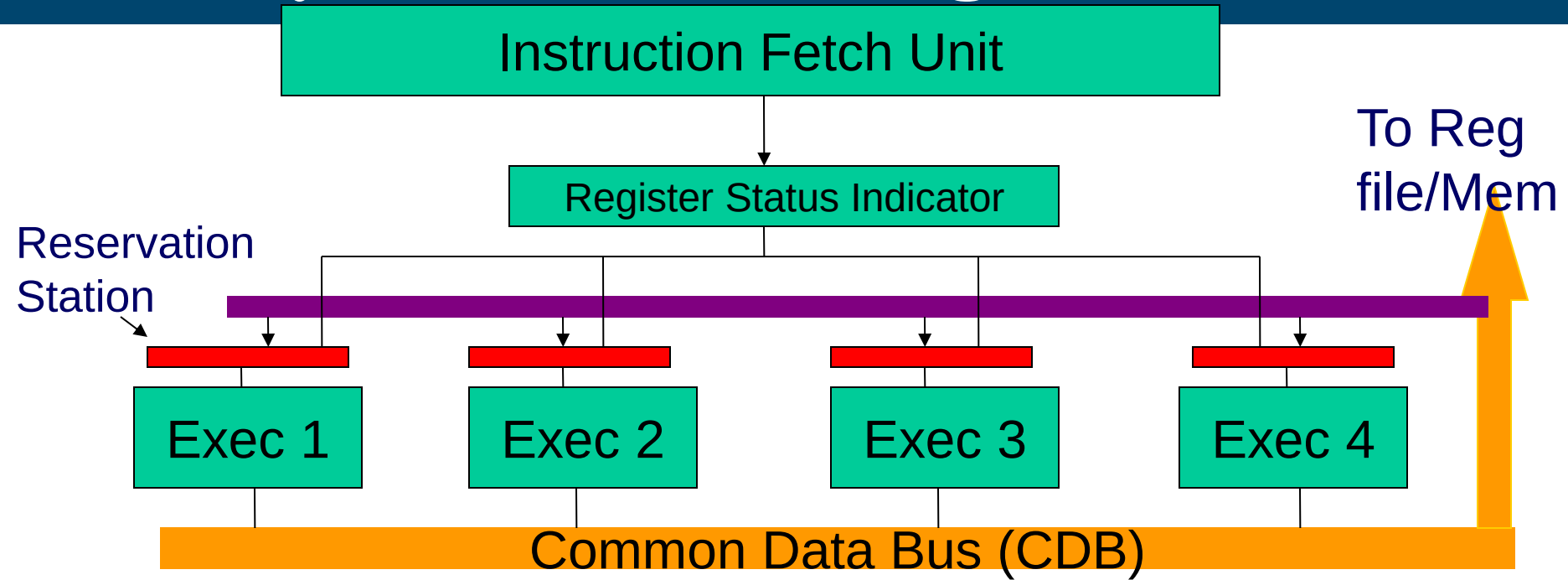
- ❖ When operand becomes available, store it in any reservation stations waiting for it
- ❖ When all operands are ready, execute the instruction
- ❖ Loads and store uses buffers
- ❖ No instruction will initiate execution until all branches that precede it in program order have completed

Dynamic Scheduling - Tomasulo

❖ Write result

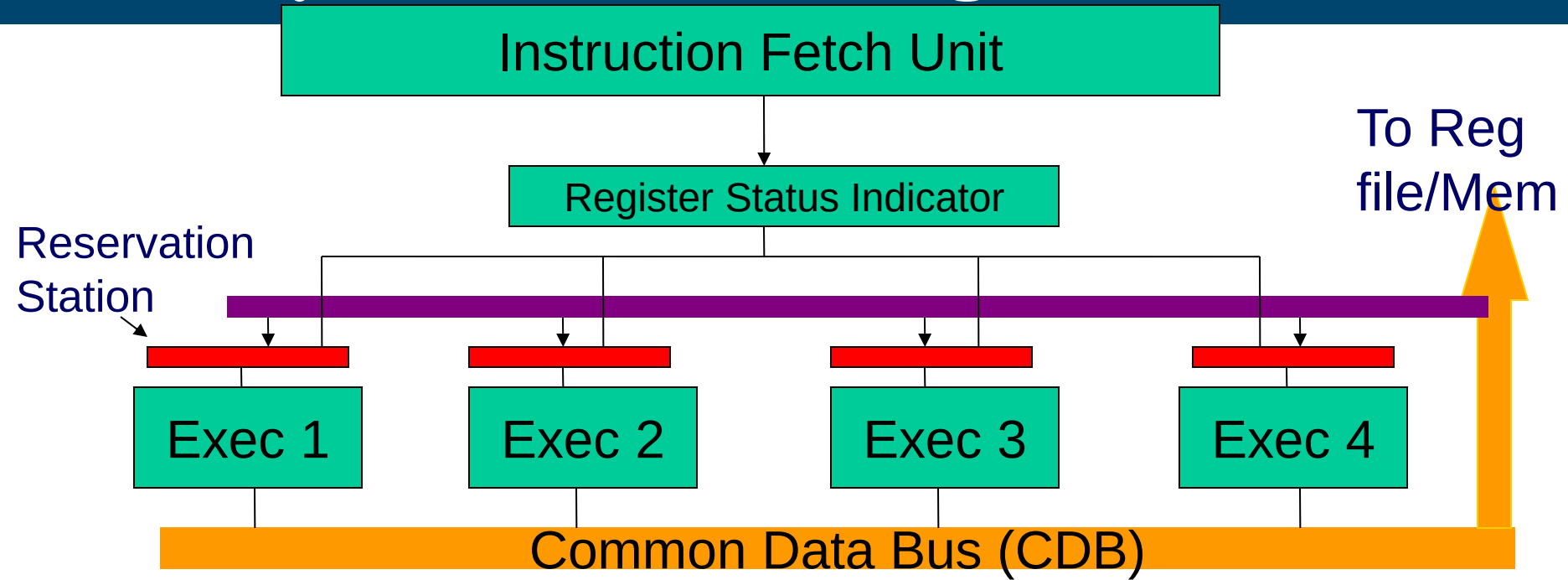
- ❖ Write result into CDB (there by it reaches the reservation station, store buffer and registers file) with name of execution unit that generated the result.
- ❖ Stores must wait until address and value are received

Dynamic Scheduling - Tomasulo



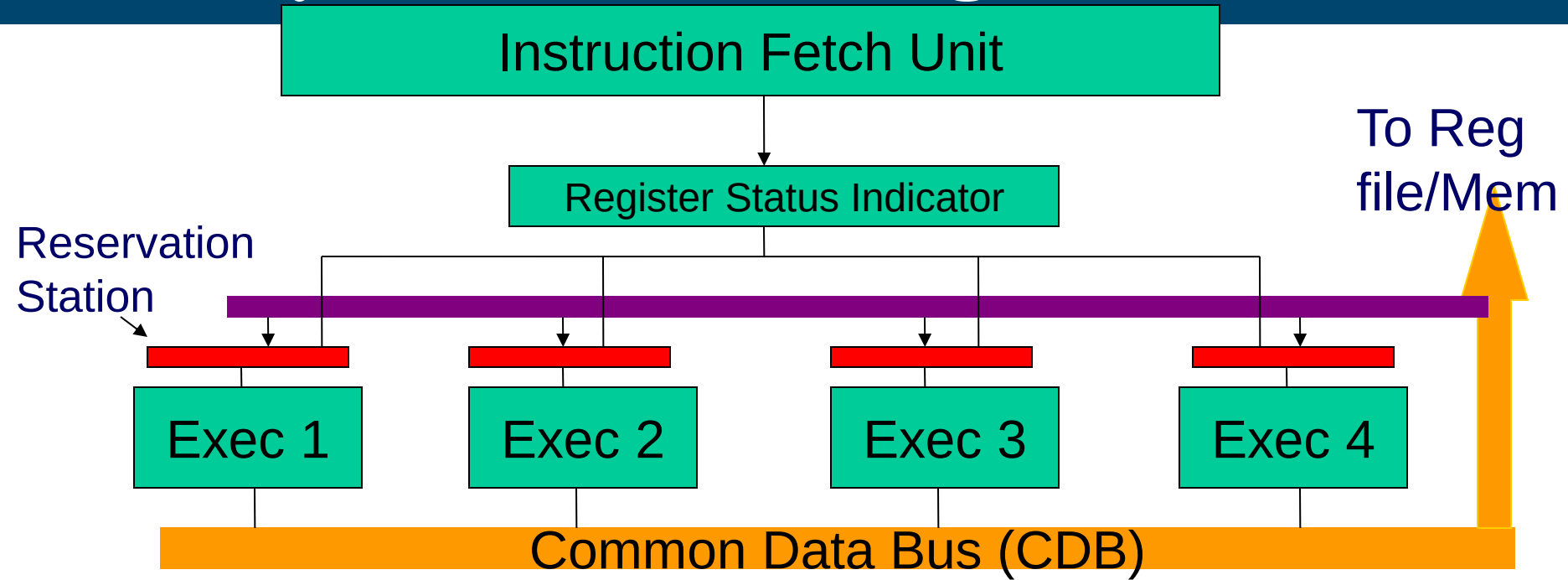
Instructions are fetched one by one and decoded to find the type of operation and the source of operands

Dynamic Scheduling - Tomasulo



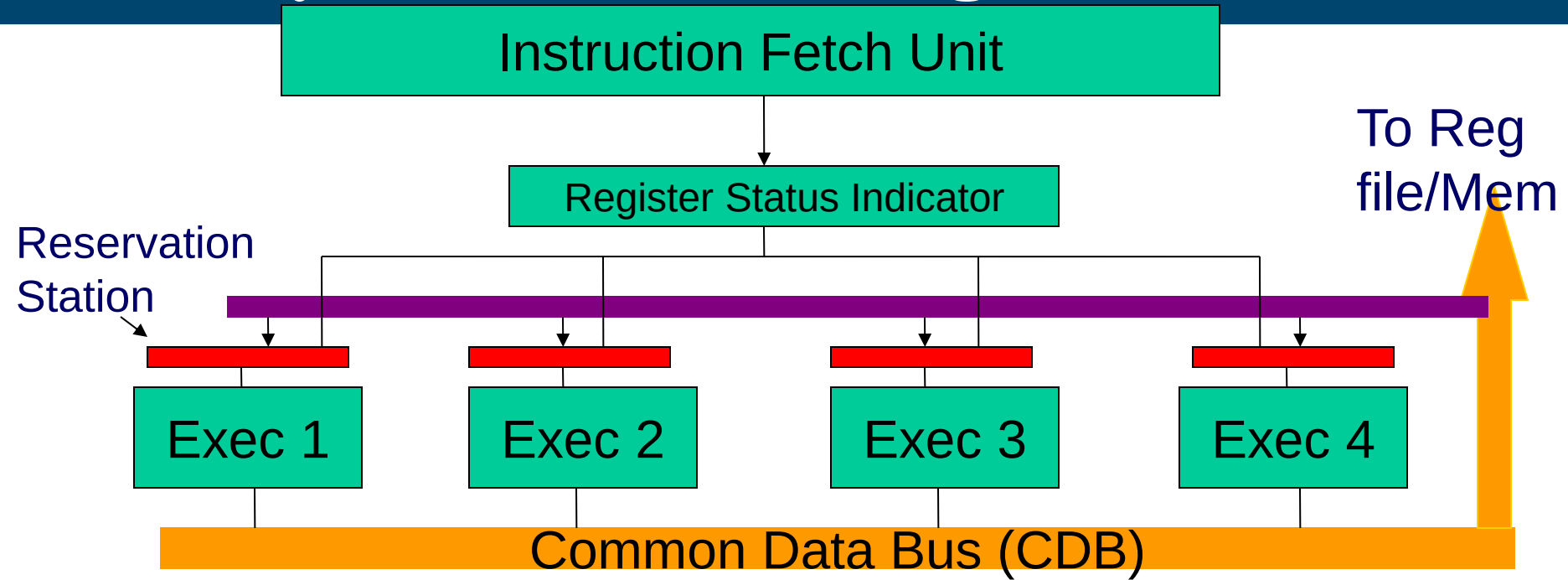
Register Status Indicator indicates whether the latest value of the register is in the reg file or currently being computed by some execution unit and if so, it states the execution unit number

Dynamic Scheduling - Tomasulo



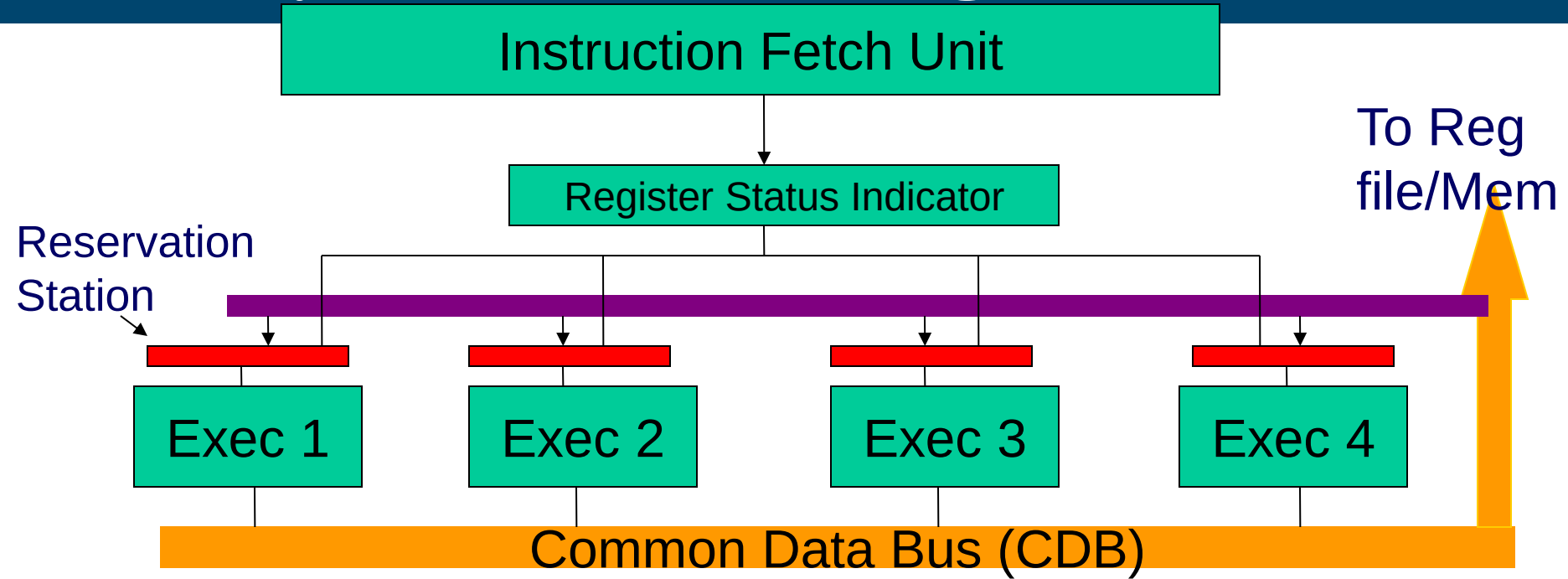
If all operands available then operation proceeds in the allotted execution unit, else, it waits in the reservation station of the allotted execution unit pinging the CDB

Dynamic Scheduling - Tomasulo



Every Execution unit writes the result along with the unit number on to the CDB which is forwarded to all reservation stations, Reg-file and Memory

Dynamic Scheduling - Tomasulo



Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch



Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	0	0	0	0	0	0	0	0	0	0

1. ADD R1, R2, R3
2. ST R1, [R4+50]
3. ADD R1, R5, R6
4. SUB R7, R1, R8
5. ST R1, [R4 + 54]
6. ADD R1, R9, R10

Empty	Empty	Empty	Empty	Empty	Empty
-------	-------	-------	-------	-------	-------

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ADD R1, R2, R3

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	1	0	0	0	0	0	0	0	0	0

1. --
2. ST R1, [R4+50]
3. ADD R1, R5, R6
4. SUB R7, R1, R8
5. ST R1, [R4 + 54]
6. ADD R1, R9, R10

Ins 1

Empty

Empty

Empty

Empty

Empty

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ST R1, [R4+50]

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	1	0	0	0	0	0	0	0	0	0

1. ---
2. ---
3. ADD R1, R5, R6
4. SUB R7, R1, R8
5. ST R1, [R4 + 54]
6. ADD R1, R9, R10

I 1, E

I 2, W 1

Empty

Empty

Empty

Empty

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ADD R1, R5, R6

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	3	0	0	0	0	0	0	0	0	0

1. ---

2. ---

3. ---

4. SUB R7, R1, R8

5. ST R1, [R4 + 54]

6. ADD R1, R9, R10

I 1, E	I 2, W 1	I 3, E	Empty	Empty	Empty
--------	----------	--------	-------	-------	-------

Note: Reservation Station stores the number of the execution unit that shall yield the latest value of a register.

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

SUB R7, R1, R8

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	3	0	0	0	0	0	4	0	0	0

1. ---
2. ---
3. ---
4. ---
5. ST R1, [R4 + 54]
6. ADD R1, R9, R10

I 1, E

I 2, W 1

I 3, E

I 4, W 3

Empty

Empty

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ST R1, [R4 + 54]

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	3	0	0	0	0	0	4	0	0	0

1. ---

2. ---

3. ---

4. ---

5. ---

6. ADD R1, R9, R10

I 1, E

I 2, W 1

I 3, E

I 4, W 3

I 5, W 3

Empty

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ADD R1, R9, R10

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	6	0	0	0	0	0	4	0	0	0

I 1, E	I 2, W 1	I 3, E	I 4, W 3	I 5, W 3	I 6, E
--------	----------	--------	----------	----------	--------

1. ADD R1, R2, R3
2. ST U1, [R4+50]
3. ADD R1, R5, R6
4. SUB R7, U3, R8
5. ST U3, [R4 + 54]
6. ADD R1, R9, R10

Effectively three Instructions are executing and others waiting for the appropriate results. The whole program is converted as shown above.

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ADD R1, R9, R10

1. ADD R1, R2, R3
2. ST U1, [R4+50]
3. ADD R1, R5, R6
4. SUB R7, U3, R8
5. ST U3, [R4 + 54]
6. ADD R1, R9, R10

Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	6	0	0	0	0	0	4	0	0	0

I 1, E	I 2, W 1	I 3, E	I 4, W 3	I 5, W 3	I 6, E
--------	----------	--------	----------	----------	--------

Operand Forwarding and Register Renaming is done automatically

Dynamic Scheduling - Tomasulo

An Example:

Instruction Fetch

ADD R1, R9, R10

1. ADD R1, R2, R3
2. ST U1, [R4+50]
3. ADD R1, R5, R6
4. SUB R7, U3, R8
5. ST U3, [R4 + 54]
6. ADD R1, R9, R10

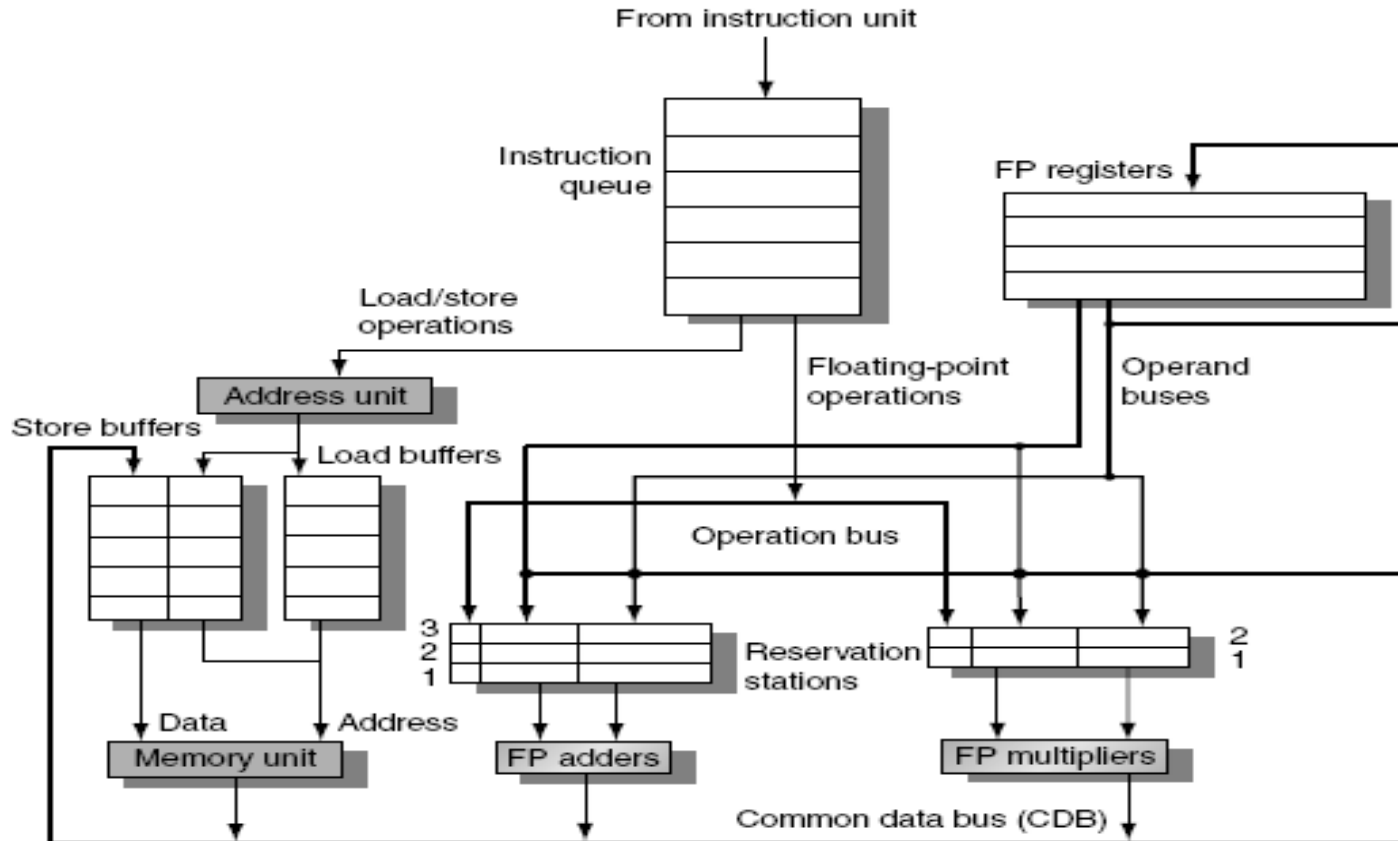
Register Status Indicator

Reg Number	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
Status	6	0	0	0	0	0	4	0	0	0

I 1, E I 2, W 1 I 3, E I 4, W 3 I 5, W 3 I 6, E

Execution unit 6, on completion will make R1 entry in Register Status Indicator 0. Similarly unit 4 will make R7 entry 0.

Tomasulo's Algorithm



Tomasulo's Algorithm for Loops

- ❖ How to run loops if no instruction will initiate execution until all branches that precede it in program order have completed?
- ❖ Predict the branches are taken and issue instructions across multiple iterations to reservation stations.

		Instruction	From Iteration
Loop:	L.D	F0,0(R1)	1
	MUL.D	F4,F0,F2	1
	S.D	F4,0(R1)	1
	DADDIU	R1,R1,-8	2
	BNE	R1,R2,Loop;	2

Instruction status				
Instruction	From Iteration	Issue	Execute	Write result
L.D F0,0(R1)	1	✓	✓	
MUL.D F4,F0,F2	1	✓		
S.D F4,0(R1)	1	✓		
L.D F0,0(R1)	2	✓	✓	
MUL.D F4,F0,F2	2	✓		
S.D F4,0(R1)	2	✓		

Load-Store Order Conflict

- ❖ A load and a store can safely be done out of order, provided they access different addresses. If a load and a store access the same address, then either
- ❖ The load is before the store in program order and interchanging them results in a WAR hazard
- ❖ The store is before the load in program order and interchanging them results in a RAW hazard.
- ❖ Interchanging two stores to the same address results in a WAW hazard.

Load-Store Order Conflict

- ❖ To allow a load and a store to interchange their order in a execution perform special address check.
- ❖ To determine if a load can be executed at a given time, the processor can check whether any uncompleted store that precedes the load in program order shares the same data memory address as the load.
- ❖ A store must wait until there are no unexecuted loads or stores that are earlier in program order that share the same data memory address.

Hardware-Based Speculation

- ❖ How to overcome control hazard in Tomasulo's approach?
- ❖ **Speculate** – **Fetch, Issue and Execute** as if branch predictions are always correct; **commit** the results only if prediction was correct
- ❖ **Instruction commit:** allowing an instruction to update the register file/memory ONLY when instruction is no longer speculative
- ❖ Need an additional hardware to prevent any irrevocable action until an instruction commits
- ❖ Reorder buffer (ROB) – holds the result of instruction between completion and commit
- ❖ Modify reservation stations- Operand source is now reorder buffer entry instead of functional unit

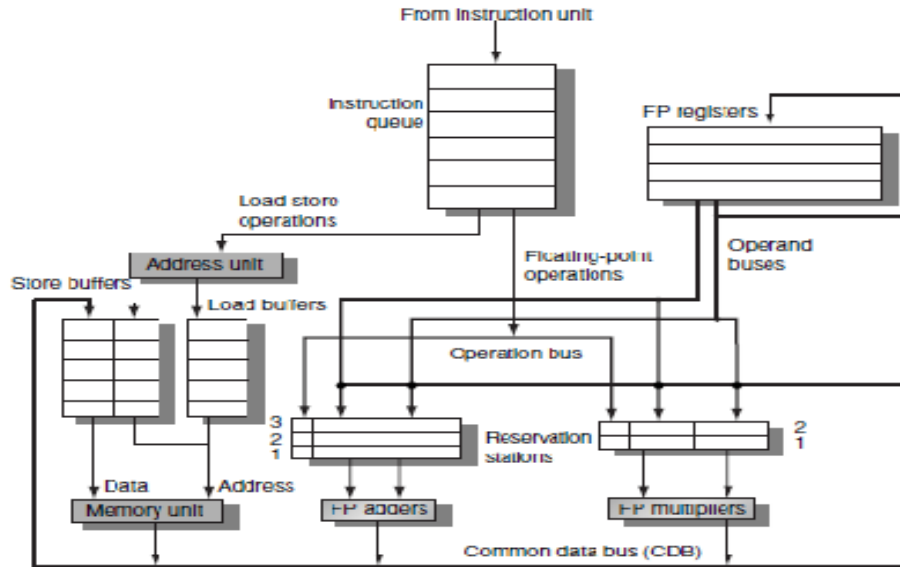
Hardware-Based Speculation

- ❖ **Hardware based speculation needs 3 techniques**
 - ❖ **Dynamic branch prediction** to choose which instructions to execute,
 - ❖ **Speculation** to allow the execution of instructions before the control dependences are resolved (with the ability to undo),
 - ❖ **Dynamic scheduling** to deal with the scheduling of different combinations of basic blocks.
- ❖ Allow an instruction to execute and to bypass its results to other instructions
- ❖ Prevent an instruction to perform any updates on writing in Reg/Mem, until the instruction is no longer speculative.

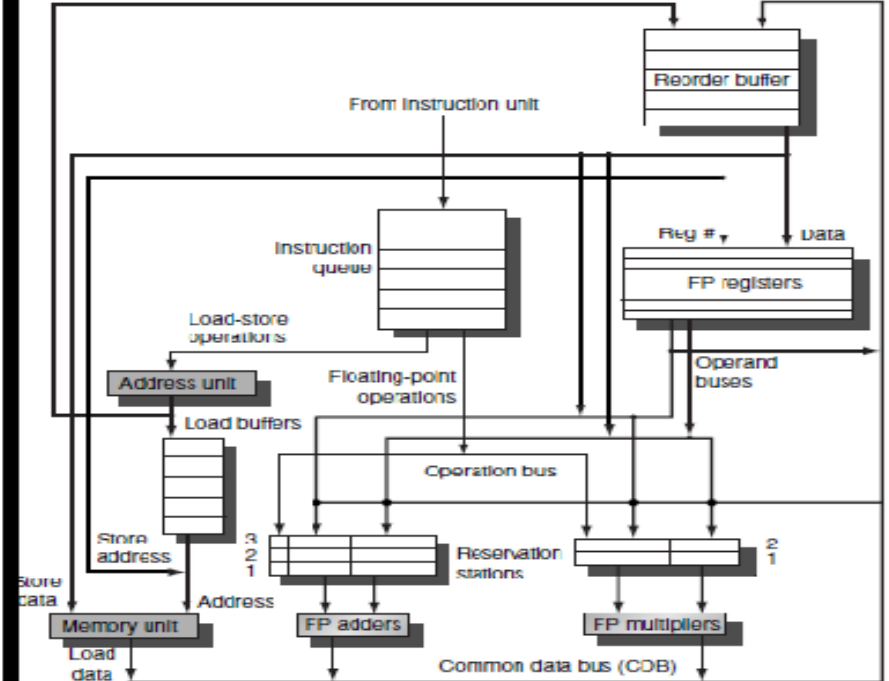
Reorder Buffer (ROB)

- ❖ Four fields:
 - ❖ Instruction type: branch/store/register
 - ❖ Destination field: register number / memory address
 - ❖ Value field: output value
 - ❖ Ready field: completed execution OR only issued
- ❖ Register values and memory values are not written until an instruction commits
- ❖ On misprediction:
 - ❖ Speculated entries in ROB are cleared

A comparison



Tomasulo's Algorithm



Tomasulo's Algorithm with ROB

Operations with ROB

- ❖ Issue
- ❖ Execute
- ❖ Write Result **(Complete)**
- ❖ Commit

Operations with ROB - Issue

- ❖ Get an instruction from the instruction queue.
- ❖ Issue the instruction if there is an empty reservation station and an empty slot in the ROB.
- ❖ Get a slot number (tag) from ROB for this instruction.
- ❖ Send the operands to the RS if they are available in either RF or in the ROB. (speculative register read)
- ❖ The number of the ROB entry is also sent to the RS to tag the result when it is placed on the CDB.
- ❖ If either all reservations are full or the ROB is full, then instruction issue is stalled until both have available entries.

Operations with ROB-Execute

- ❖ If one or more of the operands is not yet available, monitor the CDB waiting for the value.
- ❖ When both operands are available at a reservation station, execute the operation.
- ❖ Instructions may take multiple clock cycles in this stage, and loads still require two steps in this stage.
- ❖ Stores need to have the base register value available at this step for effective address calculation.

Operations with ROB– Write Result

- ❖ Write result—When the result is available, write it on the CDB (with the ROB tag).
- ❖ Data in CDB will go into the ROB and to RS waiting for the result.
- ❖ If the value to be stored is available, it is written into the Value field of the ROB entry for the store.
- ❖ If address for store is available then store that in ROB.
- ❖ If the value to be stored is not available yet, the CDB must be monitored until that value is broadcast, at which time the Value field of the ROB entry of the store is updated.

Operations with ROB - Commit

- ❖ Commit—This is the final stage of an instruction, after which only its result remains.
- ❖ Different sequences of actions at commit if instruction is
 - ❖ a branch with an incorrect prediction,
 - ❖ a store
 - ❖ other instruction (normal commit).
- ❖ The normal commit case occurs when an instruction reaches the head of the ROB and its result is present in the buffer- the processor updates the RF with the result and removes it from ROB.
- ❖ Committing a store is similar except that memory is updated rather than a result register.

Operations with ROB - Commit

- ❖ Commit—This is the final stage of an instruction, after which only its result remains.
- ❖ When a branch with incorrect prediction reaches the head of the ROB, it indicates that the speculation was wrong.
- ❖ The ROB is flushed and execution is restarted at the correct successor of the branch.
- ❖ If the branch was correctly predicted, the branch is finished.

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	No						
Add-1	No						
Add-2	No						
Add-3	No						
Mult-1	No						
Mult-2	No						

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
0	0	0	Load-1	0	0

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	Yes	Load					44+Reg[R3]
Add-1	No						
Add-2	No						
Add-3	No						
Mult-1	No						
Mult-2	No						

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
0	Load-2	0	Load-1	0	0

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	Yes	Load					44+Reg[R3]
Add-1	No						
Add-2	No						
Add-3	No						
Mult-1	Yes	MUL		Reg[F4]	Load-2		
Mult-2	No						

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
Mult-1	Load-2	0	Load-1	0	0

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	Yes	Load					44+Reg[R3]
Add-1	Yes	SUB			Load-1	Load-2	
Add-2	No						
Add-3	No						
Mult-1	Yes	MUL		Reg[F4]	Load-2		
Mult-2	No						

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
Mult-1	Load-2	0	Load-1	Add-1	0

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	Yes	Load					44+Reg[R3]
Add-1	Yes	SUB			Load-1	Load-2	
Add-2	No						
Add-3	No						
Mult-1	Yes	MUL		Reg[F4]	Load-2		
Mult-2	Yes	DIV			Mult-1	Load-1	

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
Mult-1	Load-2	0	Load-1	Add-1	Mult-2

Tomasulo's Algorithm-Illustration

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load-1	Yes	Load					32+Reg[R2]
Load-2	Yes	Load					44+Reg[R3]
Add-1	Yes	SUB			Load-1	Load-2	
Add-2	Yes	ADD			Add-1	Load-2	
Add-3	No						
Mult-1	Yes	MUL		Reg[F4]	Load-2		
Mult-2	Yes	DIV			Mult-1	Load-1	

L.D F6, 32(R2)
 L.D F2, 44(R3)
 MUL.D F0, F2, F4
 SUB.D F8, F2, F6
 DIV.D F10, F0, F6
 ADD.D F6, F8, F2

F0	F2	F4	F6	F8	F10
Mult-1	Load-2	0	Add-2	Add-1	Mult-2



johnjose@iitg.ac.in
<http://www.iitg.ac.in/johnjose/>